

Support Vector Machines (SVMs) and Kernel Methods

Daniel Wehner, M.Sc.

SAP SE / DHBW CAS Heilbronn

Summer term 2026



Find all slides on [GitHub](#) (DaWe1992/Applied_ML_Fundamentals)

Lecture Overview

- | | | | |
|-------------|-----------------------------------|-------------|------------------------------|
| I | Machine Learning Introduction | IX | Evaluation |
| II | Optimization Techniques | X | Decision Trees |
| III | Bayesian Decision Theory | • XI | Support Vector Machines |
| IV | Non-parametric Density Estimation | XII | Clustering |
| V | Probabilistic Graphical Models | XIII | Principal Component Analysis |
| VI | Linear Regression | XIV | Reinforcement Learning |
| VII | Logistic Regression | XV | Advanced Regression |
| VIII | Deep Learning | | |

Agenda for this Unit

① Linear Support Vector Machines

② Sparse Kernel Machines

③ Soft-Margin Support Vector Machines

④ Sequential Minimal Optimization (SMO)

⑤ Wrap-Up

Section:

Linear Support Vector Machines

Introduction and Motivation
Discriminant Functions and linear Separability
SVM Primal Optimization Problem
Quadratic Programming and LAGRANGE Optimization
SVM Dual Optimization Problem

What is a Support Vector Machine (SVM)?

- A **Support Vector Machine (SVM)** is a **binary classifier** introduced by VAPNIK
- SVMs are one of the most studied models, being based on statistical learning frameworks of **VC theory** proposed by VAPNIK and CHERVONENKIS
- It is a **linear** classifier

Yet another linear classifier? Unlike other linear classifiers (e. g. logistic regression), an SVM aims to maximize the margin, i. e. it maximizes the distance of the decision boundary to the closest data points from either of the classes (**maximum margin classifier**). This results in a **unique solution** for hyperplanes, and in most cases allows for better **better generalization**.

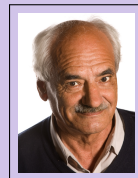


Portrait: VAPNIK and CHERVONENKIS

VLADIMIR VAPNIK, born 6 December 1936, is a computer scientist, researcher, and academic (*top image*). While at AT&T (USA), VAPNIK and his colleagues did work on the support vector machine, which he also worked on much earlier before moving to the USA. They demonstrated its performance on a number of problems of interest to the machine learning community, including handwriting recognition. Also he worked on support vector clustering algorithms.



ALEXEY CHERVONENKIS, 7 September 1938 – 22 September 2014, was a Soviet and Russian mathematician (*bottom image*). He and VAPNIK were the main developers of the VAPNIK–CHERVONENKIS (VC) theory of statistical learning, an important part of computational learning theory.



(*Wikipedia*)

Recall: Norms in $\mathbb{R}^D$

- **Definition:** Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^D$, $\alpha \in \mathbb{R}$. The mapping $\|\cdot\| : \mathbb{R}^D \rightarrow \mathbb{R}$ is called a **norm**, if the following properties hold:

$$\text{Positive definiteness:} \quad \|\mathbf{x}\| > 0 \quad \forall \mathbf{x} \in \mathbb{R}^D \setminus \{\mathbf{0}\} \quad (1)$$

$$\text{Absolute homogeneity:} \quad \|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\| \quad (2)$$

$$\text{Triangle inequality:} \quad \|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\| \quad (3)$$

- From (1) and (2) it follows $\|\mathbf{x}\| = 0 \iff \mathbf{x} = \mathbf{0}$, as $\|\mathbf{0}\| = \|0 \cdot \mathbf{0}\| = 0 \|\mathbf{0}\| = 0$
- We use $\|\cdot\|$ to denote the **EUCLIDEAN norm**: $\|\mathbf{x}\| := \sqrt{x_1^2 + \dots + x_D^2}$

Recall: Scalar Products

- **Definition:** Let $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{R}^D$ and $\alpha, \beta \in \mathbb{R}$. We call the mapping $\langle \cdot, \cdot \rangle : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ a **scalar product**, if the following properties hold:

$$\text{Positive definiteness:} \quad \langle \mathbf{x}, \mathbf{x} \rangle > 0 \quad \forall \mathbf{x} \in \mathbb{R}^D \setminus \{\mathbf{0}\} \quad (4)$$

$$\text{Symmetry:} \quad \langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle \quad (5)$$

$$\text{Linearity:} \quad \langle \alpha \mathbf{x} + \beta \mathbf{y}, \mathbf{z} \rangle = \alpha \langle \mathbf{x}, \mathbf{z} \rangle + \beta \langle \mathbf{y}, \mathbf{z} \rangle \quad (6)$$

- With this definition we can write $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} \iff \|\mathbf{x}\|^2 = \langle \mathbf{x}, \mathbf{x} \rangle$
- $\langle \mathbf{x}, \mathbf{y} \rangle \equiv \mathbf{x}^\top \mathbf{y}$ is the **canonical scalar product**



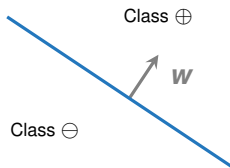
Discriminant Functions

Assume we have an **affine-linear function** which defines the **decision boundary**:

$$f(\mathbf{x}) := \langle \mathbf{w}, \mathbf{x} \rangle + b \quad (7)$$

Remarks:

- $\mathbf{w} \in \mathbb{R}^M$ are the **weights**, and $b \in \mathbb{R}$ is called **bias**
- For $\mathbf{x} \in \mathbb{R}^M$ on the hyperplane, we have $f(\mathbf{x}) = 0$
- An input vector $\mathbf{x}$ is assigned...
 - ...to the positive class $\oplus$, if $f(\mathbf{x}) \geq 0$
 - ...to the negative class $\ominus$, if $f(\mathbf{x}) < 0$



Linear Separability

- Let a dataset $\mathcal{D} := \{(\mathbf{x}^n, y_n)\}_{n=1}^N$ be given, where $\mathbf{x}^n \in \mathbb{R}^M$, $y_n \in \{-1, +1\}$
- A new data point $\mathbf{x}'$ is classified according to the sign of $f(\mathbf{x}')$:

$$\hat{y} := h_{\mathbf{w},b}(\mathbf{x}') := \text{sign}(f(\mathbf{x}')) \quad (8)$$

- The sign-function is defined as

$$\text{sign}(z) := \begin{cases} -1 & \text{if } z < 0 \\ +1 & \text{if } z \geq 0 \end{cases} \quad (9)$$

Linear Separability (Ctd.)

- A dataset is called **linearly separable** in feature space, if there exist $\mathbf{w} \in \mathbb{R}^M$ and $b \in \mathbb{R}$, such that

$$f(\mathbf{x}^n) = \langle \mathbf{w}, \mathbf{x}^n \rangle + b \geq 0 \quad \forall \mathbf{x}^n \text{ with } y_n = +1 \quad (10)$$

$$f(\mathbf{x}^n) = \langle \mathbf{w}, \mathbf{x}^n \rangle + b < 0 \quad \forall \mathbf{x}^n \text{ with } y_n = -1 \quad (11)$$

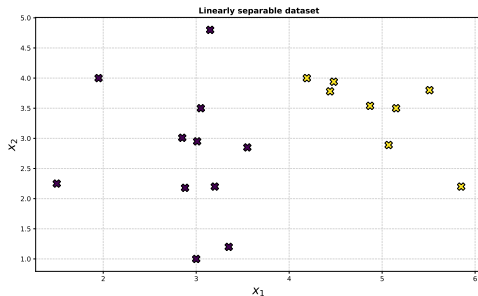
- We can write this in a more compact form:

Linear separability of a dataset:

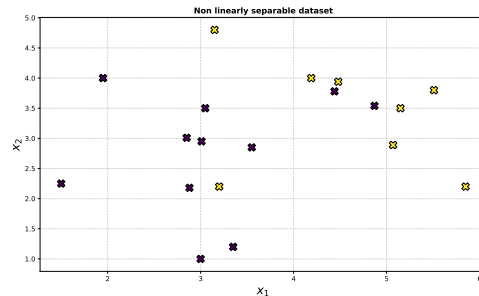
$$y_n f(\mathbf{x}^n) \geq 0 \quad \forall n = 1, \dots, N \quad (12)$$

(Not) linearly separable Data

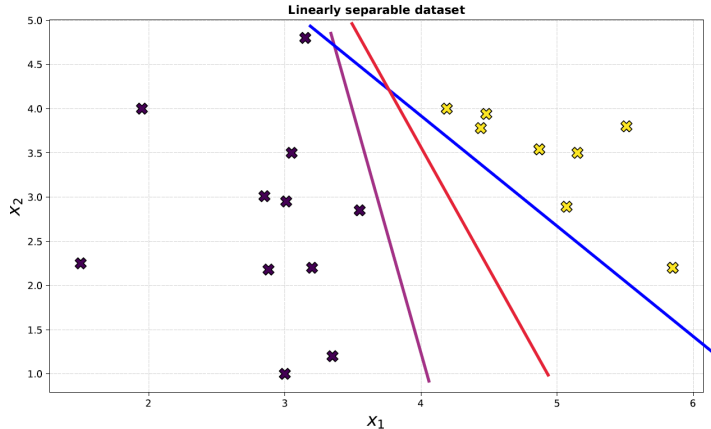
Linearly separable:



Not linearly separable:



Which Decision Boundary is the best?

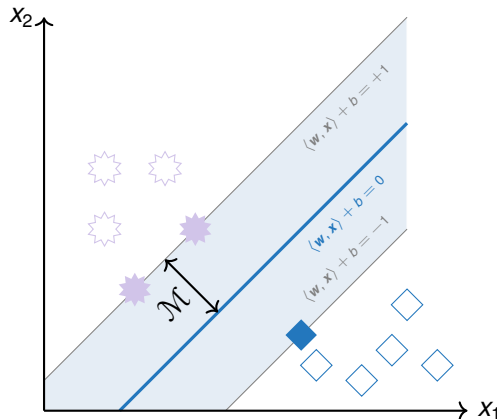


Maximum Margin Classifiers

- An SVM maximizes the **margin** $\mathcal{M}$

$$\mathcal{M}^* := \max_{w,b} \mathcal{M}$$

- The larger $\mathcal{M}^*$, the less likely are false predictions $\Rightarrow$ **better generalization**
- The decision boundary is fully determined by the set of **support vectors** $\mathcal{S}$ (*filled data points*)





(Orthogonal) Projection of Vectors

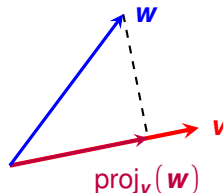
- The length of the vector $\text{proj}_{\mathbf{v}}(\mathbf{w})$ is given by (*trigonometry!*)

$$\|\text{proj}_{\mathbf{v}}(\mathbf{w})\| = \|\mathbf{w}\| \cos \angle(\mathbf{v}, \mathbf{w})$$

$$= \|\mathbf{w}\| \frac{\langle \mathbf{v}, \mathbf{w} \rangle}{\|\mathbf{v}\| \cdot \|\mathbf{w}\|} = \frac{\langle \mathbf{v}, \mathbf{w} \rangle}{\|\mathbf{v}\|}$$

- Multiply the length with a unit vector pointing in the direction of $\mathbf{v}$:

$$\text{proj}_{\mathbf{v}}(\mathbf{w}) = \|\text{proj}_{\mathbf{v}}(\mathbf{w})\| \cdot \frac{\mathbf{v}}{\|\mathbf{v}\|} = \frac{\langle \mathbf{v}, \mathbf{w} \rangle}{\|\mathbf{v}\|^2} \mathbf{v} \quad (13)$$





Weight Vector is perpendicular to the Hyperplane

Lemma: The weight vector $\mathbf{w}$ is perpendicular to the hyperplane defined by the discriminant function f .

Proof: Let $\mathbf{x} \in \mathbb{R}^M$ and $\mathbf{y} \in \mathbb{R}^M$ be two distinct points on the decision surface. Then by definition we have

$$f(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b = 0 \quad \text{and} \quad f(\mathbf{y}) = \langle \mathbf{w}, \mathbf{y} \rangle + b = 0. \quad (14)$$

This implies $\langle \mathbf{w}, \mathbf{x} \rangle = \langle \mathbf{w}, \mathbf{y} \rangle$ which is equivalent to $\langle \mathbf{w}, \mathbf{x} - \mathbf{y} \rangle = 0$ (we have used equation (6) here). The vector $\mathbf{x} - \mathbf{y}$ is entirely contained in the hyperplane and the dot product with $\mathbf{w}$ is equal to zero. Thus, $\mathbf{w}$ is orthogonal to the decision boundary defined by f . ■



Distance of a Point to the Hyperplane

Lemma (Distance of a point to the hyperplane): The (perpendicular) distance $d \in \mathbb{R}$ of a point $\mathbf{p} \in \mathbb{R}^M$ to the hyperplane is given by

$$d := \frac{|f(\mathbf{p})|}{\|\mathbf{w}\|}. \quad (15)$$

Furthermore, we obtain the **signed distance** $d_s \in \mathbb{R}$ of $\mathbf{p}$ to the hyperplane by replacing $|f(\mathbf{p})|$ with $f(\mathbf{p})$ in equation (15). The sign of d_s indicates whether the data point is located on the left side or on right side of the hyperplane.



Distance of a Point to the Hyperplane – Proof

Proof: Let $\mathbf{p} \in \mathbb{R}^M$ be an arbitrary point and $\mathbf{p}_\perp \in \mathbb{R}^M$ its orthogonal projection onto the decision hyperplane. We notice that $\mathbf{p} = \mathbf{p}_\perp + d_s \frac{\mathbf{w}}{\|\mathbf{w}\|}$, where d_s is the (signed) perpendicular distance of $\mathbf{p}$ to the hyperplane whose normal vector is $\mathbf{w}$. Hence,

$$f(\mathbf{p}) = \langle \mathbf{w}, \mathbf{p} \rangle + b = \left\langle \mathbf{w}, \mathbf{p}_\perp + d_s \frac{\mathbf{w}}{\|\mathbf{w}\|} \right\rangle + b \stackrel{(6)}{=} \langle \mathbf{w}, \mathbf{p}_\perp \rangle + b + d_s \frac{\langle \mathbf{w}, \mathbf{w} \rangle}{\|\mathbf{w}\|}. \quad (16)$$

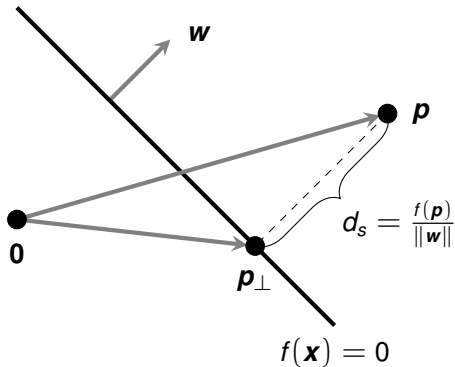
We rearrange this equation for d_s and exploit $\langle \mathbf{w}, \mathbf{p}_\perp \rangle + b = f(\mathbf{p}_\perp) = 0$ (because $\mathbf{p}_\perp$ lies on the decision boundary) as well as $\langle \mathbf{w}, \mathbf{w} \rangle = \|\mathbf{w}\|^2$. We obtain

$$d_s = \frac{f(\mathbf{p})}{\|\mathbf{w}\|}.$$

Finally we have $d = |d_s| = |f(\mathbf{p})|/\|\mathbf{w}\|$ and the proof is complete. ■



Distance of a Point to the Hyperplane – Proof (Ctd.)





Distance of a Point to the Hyperplane – Another Proof

Proof: Let $\mathbf{p} \in \mathbb{R}^M$ be the point for which we wish to calculate the distance to the hyperplane. Let further $\mathbf{x} \in \mathbb{R}^M$, $\mathbf{q} \in \mathbb{R}^M$ be two points on the decision boundary. Specifically, we have $f(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b = 0$. Furthermore, $\mathbf{x} - \mathbf{q}$ is entirely contained in the hyperplane. Thus, we have

$$\langle \mathbf{w}, \mathbf{x} - \mathbf{q} \rangle = 0 \stackrel{(6)}{\iff} \langle \mathbf{w}, \mathbf{x} \rangle - \langle \mathbf{w}, \mathbf{q} \rangle = 0 \iff \langle \mathbf{w}, \mathbf{x} \rangle = \langle \mathbf{w}, \mathbf{q} \rangle. \quad (17)$$

Equation (17) can be rewritten to $w_1 x_1 + \dots + w_M x_M = \langle \mathbf{w}, \mathbf{q} \rangle = -b$. We now define $\mathbf{z} := \mathbf{p} - \mathbf{q}$ and compute the orthogonal projection of $\mathbf{z}$ onto the normal vector $\mathbf{w}$:

$$\text{proj}_{\mathbf{w}}(\mathbf{z}) \stackrel{(13)}{=} \frac{\langle \mathbf{w}, \mathbf{z} \rangle}{\|\mathbf{w}\|^2} \mathbf{w} = \frac{\langle \mathbf{w}, \mathbf{p} - \mathbf{q} \rangle}{\|\mathbf{w}\|^2} \mathbf{w} \stackrel{(6)}{=} \frac{\langle \mathbf{w}, \mathbf{p} \rangle - \langle \mathbf{w}, \mathbf{q} \rangle}{\|\mathbf{w}\|^2} \mathbf{w} = \frac{\langle \mathbf{w}, \mathbf{p} \rangle + b}{\|\mathbf{w}\|^2} \mathbf{w} \quad (18)$$

Distance of a Point to the Hyperplane – Another Proof (Ctd.)

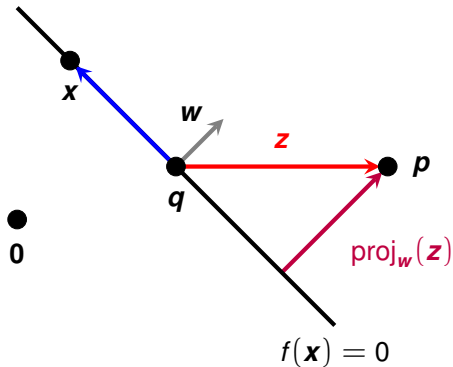
The distance d is given by the norm of $\text{proj}_{\mathbf{w}}(\mathbf{z})$, i. e.

$$\begin{aligned} d &= \|\text{proj}_{\mathbf{w}}(\mathbf{z})\| = \left\| \frac{\langle \mathbf{w}, \mathbf{p} \rangle + b}{\|\mathbf{w}\|^2} \mathbf{w} \right\| \stackrel{(2)}{=} \frac{|\langle \mathbf{w}, \mathbf{p} \rangle + b|}{\|\mathbf{w}\|^2} \cdot \|\mathbf{w}\| \\ &= \frac{|\langle \mathbf{w}, \mathbf{p} \rangle + b|}{\|\mathbf{w}\|} \\ &= \frac{|f(\mathbf{p})|}{\|\mathbf{w}\|}, \end{aligned} \tag{19}$$

as claimed. 



Distance of a Point to the Hyperplane – Another Proof (Ctd.)





Distance of the Hyperplane to the Origin

Lemma (Distance to the origin): The distance of the hyperplane to the origin is given by

$$d_0 := -\frac{b}{\|\mathbf{w}\|}. \quad (20)$$

Proof: Let $\mathbf{x} \in \mathbb{R}^M$ be a point on the hyperplane. The distance of $\mathbf{x}$ to the hyperplane is then

$$\frac{\langle \mathbf{w}, \mathbf{x} \rangle + b}{\|\mathbf{w}\|} = 0 \quad \Longleftrightarrow \quad \frac{\langle \mathbf{w}, \mathbf{x} \rangle}{\|\mathbf{w}\|} = -\frac{b}{\|\mathbf{w}\|}.$$

The left-hand side is the scalar projection of $\mathbf{x}$ onto $\mathbf{w}$, which is the distance to the origin. ■

Derivation of the SVM Primal Optimization Problem

- Let a **linearly separable** dataset $\mathcal{D}$ be given
- From equation (15) we know that the perpendicular distance of a point $\mathbf{x}$ to the hyperplane defined by $f(\mathbf{x}) = 0$ is given by

$$d = \frac{|f(\mathbf{x})|}{\|\mathbf{w}\|} = \frac{|\langle \mathbf{w}, \mathbf{x} \rangle + b|}{\|\mathbf{w}\|} \quad (21)$$

- We are only interested in solutions for which all data points are correctly classified, i. e. $y_n f(\mathbf{x}^n) \geq 0$ for all $n = 1, \dots, N$
- Such a solution exists, otherwise the dataset would **not** be linearly separable

Derivation of the SVM Primal Optimization Problem (Ctd.)

- Thus, the distance d is given by:

$$d = \frac{y_n f(\mathbf{x}^n)}{\|\mathbf{w}\|} = \frac{y_n (\langle \mathbf{w}, \mathbf{x}^n \rangle + b)}{\|\mathbf{w}\|} \quad (22)$$

- The **margin** $\mathcal{M}$ is given by the distance of the **closest data point** to the decision surface (*we call this data point $\tilde{\mathbf{x}}$*)

Goal:

We wish to optimize the SVM parameters w and b so as to maximize this distance (maximum margin)

Derivation of the SVM Primal Optimization Problem (Ctd.)

Therefore we have to solve:

SVM optimization problem:

$$(\mathbf{w}^*, b^*) := \arg \max_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \min_i \left\{ y_i (\langle \mathbf{w}, \mathbf{x}^i \rangle + b) \right\} \right\} \quad (23)$$

Problem: A direct solution to the optimization problem (23) would be very hard to obtain! We shall therefore rewrite this optimization problem to obtain a solution more easily

Derivation of the SVM Primal Optimization Problem (Ctd.)

- We notice that rescaling $\mathbf{w}$ and b by a factor κ **does not change the distance** to the decision boundary (*we can cancel κ*)
- Therefore, we can choose κ so that

$$\tilde{y}(\langle \mathbf{w}, \tilde{\mathbf{x}} \rangle + b) = 1 \quad (24)$$

for the data point $\tilde{\mathbf{x}}$ (and corresponding label $\tilde{y}$) closest to the decision surface

- All data points $\mathbf{x}^n$ ($n = 1, \dots, N$) then satisfy the constraint:

$$y_n(\langle \mathbf{w}, \mathbf{x}^n \rangle + b) \geq 1 \quad (25)$$



Calculation of the Margin

- Let $\tilde{\mathbf{x}}$ be the data point closest to the decision boundary
- The margin is defined as the perpendicular distance of $\tilde{\mathbf{x}}$ to the boundary
- We plug equation (24) into equation (22) to obtain the margin

Lemma (Margin): The margin $\mathcal{M}$ is the distance of the closest data point to the decision boundary. It is given by

$$\mathcal{M} := \frac{1}{\|\mathbf{w}\|}. \quad (26)$$



Primal Optimization Problem for SVMs

We substitute the equations (24) and (25) into equation (23) to obtain the

Primal optimization problem for SVMs:

$$\text{minimize} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad (27)$$

$$\text{subject to} \quad y_n (\langle \mathbf{w}, \mathbf{x}^n \rangle + b) - 1 \geq 0 \quad (n = 1, \dots, N) \quad (28)$$

- Instead of maximizing $\frac{1}{\|\mathbf{w}\|}$, we choose to minimize $\|\mathbf{w}\|^2$
- The factor $1/2$ was added for later mathematical convenience

Quadratic Programming (QP)

- This is a **quadratic programming (QP)** problem
- The objective function is a quadratic function of the parameters $\mathbf{w}$:

$$q(\mathbf{w}) := \frac{1}{2} \|\mathbf{w}\|^2$$

- The constraints (**one for each data point!**) are linear:

$$g_n(\mathbf{w}, b) := -y_n(\langle \mathbf{w}, \mathbf{x}^n \rangle + b) + 1 \leq 0$$

- A global optimum is guaranteed to exist due to the **convexity** of q

Primal and dual Optimization Problems

- We will derive the dual optimization problem for SVMs which has some benefits
- Consider the **primal optimization problem**

$$\begin{array}{ll} \text{minimize} & q(\mathbf{w}) \\ \text{subject to} & g_n(\mathbf{w}, b) \leq 0 \quad (n = 1, \dots, N). \end{array}$$

- The **dual optimization problem** is then given by

$$\begin{array}{ll} \text{maximize (!)} & \mathcal{D}(\alpha) := \inf_{\mathbf{w} \in \mathbb{R}^M, b \in \mathbb{R}} \mathcal{L}(\mathbf{w}, b, \alpha) \\ \text{subject to} & \alpha \geq \mathbf{0}. \end{array}$$

LAGRANGE Function

- The **LAGRANGE function** (also called *Lagrangian*) is defined as:

$$\mathcal{L}(\mathbf{w}, b, \alpha) := q(\mathbf{w}) + \sum_{n=1}^N \alpha_n g_n(\mathbf{w}, b)$$

- Since $\mathcal{L}$ is convex, we can compute $\inf_{\mathbf{w} \in \mathbb{R}^M, b \in \mathbb{R}} \mathcal{L}(\mathbf{w}, b, \alpha)$ by considering

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}, b, \alpha) \stackrel{!}{=} \mathbf{0} \quad \text{and} \quad \nabla_b \mathcal{L}(\mathbf{w}, b, \alpha) \stackrel{!}{=} 0$$

- We then solve for the minimizing primal variables $\mathbf{w}^*$ and b^* and plug them into $\mathcal{L}$ (or add a constraint to the dual problem if this is not possible)
- The result is the dual function $\mathcal{D}$ which only depends on the multipliers α



SVM LAGRANGE Function

We plug in the function definitions into $\mathcal{L}$ and obtain the

LAGRANGE function for SVMs:

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{n=1}^N \alpha_n \left[y_n (\langle \mathbf{w}, \mathbf{x}^n \rangle + b) - 1 \right] \quad (29)$$

Later we shall see that the LAGRANGE multipliers α_n will be non-zero for all support vectors, all other multipliers will turn out to be zero



Gradients of the LAGRANGE Function

We compute the gradients of $\mathcal{L}$ w. r. t. $\mathbf{w}$ and b and set them to zero:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}, b, \alpha) = \mathbf{w} - \sum_{n=1}^N \alpha_n y_n \mathbf{x}^n \stackrel{!}{=} \mathbf{0} \quad \Longrightarrow \quad \boxed{\mathbf{w} = \sum_{n=1}^N \alpha_n y_n \mathbf{x}^n} \quad (30)$$

$$\nabla_b \mathcal{L}(\mathbf{w}, b, \alpha) = - \sum_{n=1}^N \alpha_n y_n \stackrel{!}{=} 0 \quad \Longrightarrow \quad \boxed{\sum_{n=1}^N \alpha_n y_n = 0} \quad (31)$$

From equation (30) we see that $\mathbf{w}$ is a linear combination of the input!

Dual Optimization Problem for SVMs

Now we plug equations (30) and (31) into the LAGRANGE function (29) to obtain:

$$\begin{aligned}
 \mathcal{D}(\alpha) &= \frac{1}{2} \left\langle \sum_{i=1}^N \alpha_i y_i \mathbf{x}^i, \sum_{j=1}^N \alpha_j y_j \mathbf{x}^j \right\rangle - \sum_{i=1}^N \alpha_i \left[y_i \left(\left\langle \sum_{j=1}^N \alpha_j y_j \mathbf{x}^j, \mathbf{x}^i \right\rangle + b \right) - 1 \right] \\
 &\stackrel{(6)}{=} \frac{1}{2} \left\langle \sum_{i=1}^N \alpha_i y_i \mathbf{x}^i, \sum_{j=1}^N \alpha_j y_j \mathbf{x}^j \right\rangle - \left\langle \sum_{i=1}^N \alpha_i y_i \mathbf{x}^i, \sum_{j=1}^N \alpha_j y_j \mathbf{x}^j \right\rangle \\
 &\quad - \underbrace{\sum_{i=1}^N \alpha_i y_i b}_{= 0 \Rightarrow (31)} + \sum_{i=1}^N \alpha_i
 \end{aligned} \tag{32}$$

Dual Optimization Problem for SVMs (Ctd.)

$$\begin{aligned}
 &= \sum_{i=1}^N \alpha_i - \frac{1}{2} \left\langle \sum_{i=1}^N \alpha_i y_i \mathbf{x}^i, \sum_{j=1}^N \alpha_j y_j \mathbf{x}^j \right\rangle \\
 &\stackrel{(6)}{=} \boxed{\sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \langle \mathbf{x}^i, \mathbf{x}^j \rangle} \tag{33}
 \end{aligned}$$

- We have found the dual objective function $\mathfrak{D}$
- The dual optimization problem has the constraints $\alpha \geq \mathbf{0}$ as well as the constraint given by equation (31): $\sum_{i=1}^N \alpha_i y_i = 0$



Dual Optimization Problem for SVMs (Ctd.)

Dual optimization problem for SVMs (WOLFE dual):

$$\text{maximize} \quad \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \langle \mathbf{x}^i, \mathbf{x}^j \rangle \quad (34)$$

$$\text{subject to} \quad \alpha_i \geq 0 \text{ for all } i = 1, \dots, N \quad (35)$$

$$\sum_{i=1}^N \alpha_i y_i = 0 \quad (36)$$

Some Remarks

- The solution to a quadratic programming problem in D variables has **computational complexity** of $\mathcal{O}(D)$ in general
- The primal SVM optimization problem had M variables (# features), whereas the dual representation has N variables (# data points)

Considering $M \ll N$, the dual representation seems disadvantageous

However, the dual representation of the optimization problem unlocks the concept of kernels (*see next section*)

Computation of the Offset b

- Once we know α , we can determine b by noting that any support vector $\mathbf{x}$ satisfies (where $\mathcal{S}$ is the set of indices of support vectors):

$$yf(\mathbf{x}) = y(\langle \mathbf{x}, \mathbf{w} \rangle + b) \stackrel{(30),(6)}{=} y \left(\sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}, \mathbf{x}^j \rangle + b \right) = 1 = y^2 \quad (37)$$

- Rearrange the above equation for b and average over all support vectors to compute b :

$$b := \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(y_i - \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}^i, \mathbf{x}^j \rangle \right) \quad (38)$$



SVM Decision Rule

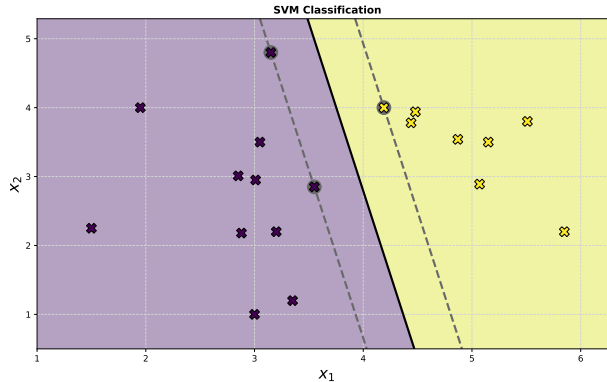
- Given our derivations, we can rewrite the SVM decision rule as follows:

$$h(\mathbf{x}') := \text{sign}(\langle \mathbf{w}, \mathbf{x}' \rangle + b) \stackrel{(30),(6)}{=} \text{sign} \left(\sum_{i \in \mathcal{S}} \alpha_i y_i \langle \mathbf{x}^i, \mathbf{x}' \rangle + b \right) \quad (39)$$

- $\mathbf{x}'$ is an unknown instance for which the class label is not known

Since all α_i will be zero for non-support vectors, the decision for a class depends on the support vectors only! This makes predictions fast, even for large datasets. The number of support vectors can also be used as an evaluation criterion.

Example: Linear SVM



Section:

Sparse Kernel Machines

Feature Mapping / Disadvantages of Feature Mapping

Introduction to the Kernel Method

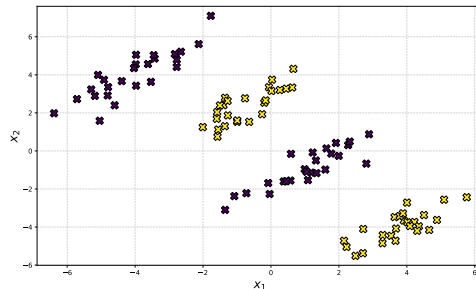
The Kernel Matrix

MERCER's Condition and MERCER Kernels

Application of Kernels to SVMs

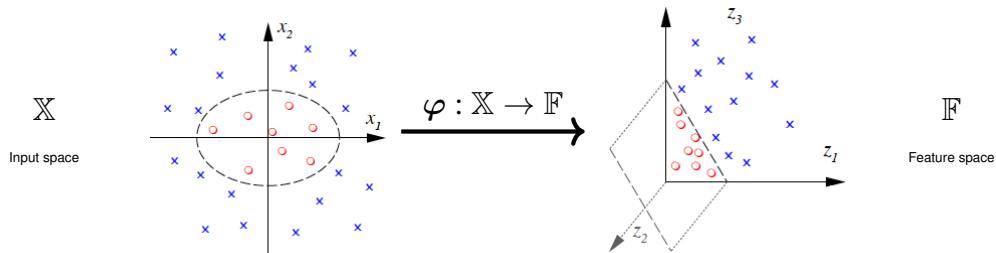
Non-Linear SVMs / Non-Linear Separability

- So far we have assumed **linear separability** of the data
- What if the data is **not linearly separable**?
- We cannot find a straight line to separate the data
- We have already learned about the **feature mapping** technique



Feature Mapping

The mapping function φ maps from input space $\mathbb{X}$ to feature space $\mathbb{F}$:



$$\varphi(x_1, x_2) = \left(x_1^2, \sqrt{2}x_1x_2, x_2^2 \right) =: (z_1, z_2, z_3)$$

Feature Mapping (Ctd.)

Disadvantages of feature mapping:

- When using feature maps, we **EXPLICITLY** transform the data to a higher dimension where we hope classification becomes easier
- Computing the feature map can become very expensive
- And how do we know how many and which dimensions to add?

Alternative: In this section we will introduce the **kernel concept** which can be used as an alternative to feature mapping (*if certain prerequisites are met*)



Kernel Methods and the Kernel Trick

Kernel methods owe their name to the use of **kernel functions**, which enable them to operate in a high-dimensional (*potentially even infinite-dimensional*), **IMPLICIT** feature space **without ever computing the coordinates of the data in that space**, but rather by simply **computing the inner products between the images of all pairs of data in the feature space**.

This operation is often computationally cheaper than the explicit computation of the coordinates (*also known as feature mapping*). This approach is referred to as the **kernel trick**.

(Wikipedia)



What is a Kernel Function?

- Let $\mathbb{X}$ be the input space and $\mathbb{F}$ the higher-dimensional feature space
- A kernel function k can be considered a **similarity function**, i. e. $k(\mathbf{x}, \tilde{\mathbf{x}})$ is a measure of how similar $\mathbf{x} \in \mathbb{X}$ and $\tilde{\mathbf{x}} \in \mathbb{X}$ are in feature space $\mathbb{F}$

A **kernel function** $k : \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{R}$ maps a pair of input features to a real number. We can express k in terms of a feature map:

$$k(\mathbf{x}, \tilde{\mathbf{x}}) := \langle \varphi(\mathbf{x}), \varphi(\tilde{\mathbf{x}}) \rangle_{\mathbb{F}} \quad (40)$$

with $\varphi : \mathbb{X} \rightarrow \mathbb{F}$, i. e. k must be **symmetric** and **positive-semidefinite**.

Inner Products measure Distances!

Consider distances in the transformed feature space $\mathbb{F}$:

$$\begin{aligned}\|\varphi(\mathbf{x}) - \varphi(\tilde{\mathbf{x}})\|_{\mathbb{F}}^2 &= \langle \varphi(\mathbf{x}) - \varphi(\tilde{\mathbf{x}}), \varphi(\mathbf{x}) - \varphi(\tilde{\mathbf{x}}) \rangle_{\mathbb{F}} \\ &= \langle \varphi(\mathbf{x}), \varphi(\mathbf{x}) \rangle_{\mathbb{F}} - 2\langle \varphi(\mathbf{x}), \varphi(\tilde{\mathbf{x}}) \rangle_{\mathbb{F}} + \langle \varphi(\tilde{\mathbf{x}}), \varphi(\tilde{\mathbf{x}}) \rangle_{\mathbb{F}}\end{aligned}$$

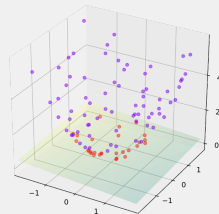
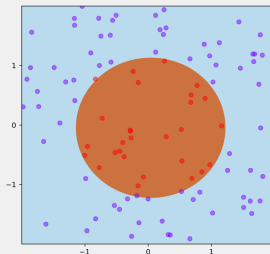
Conclusion: Distances can be computed by evaluating inner products!

Example: Kernel Function

Let the following feature map be given:

$$\varphi: \mathbb{R}^2 \rightarrow \mathbb{R}^3, (x_1, x_2)^\top \mapsto (x_1, x_2, x_1^2 + x_2^2)^\top \quad (41)$$

Visualization:



Example: Kernel Function (Ctd.)

The kernel function k corresponding to the feature map given in (41) is

$$k(\mathbf{x}, \tilde{\mathbf{x}}) = \langle \mathbf{x}, \tilde{\mathbf{x}} \rangle_{\mathbb{X}} + \|\mathbf{x}\|_{\mathbb{X}}^2 \cdot \|\tilde{\mathbf{x}}\|_{\mathbb{X}}^2. \quad (42)$$

Proof: Let $\mathbf{x} = (x_1, x_2)^{\top} \in \mathbb{R}^2$ and $\tilde{\mathbf{x}} = (\tilde{x}_1, \tilde{x}_2)^{\top} \in \mathbb{R}^2$. Then:

$$\begin{aligned} \langle \varphi(\mathbf{x}), \varphi(\tilde{\mathbf{x}}) \rangle_{\mathbb{F}} &= \left\langle (x_1, x_2, x_1^2 + x_2^2), (\tilde{x}_1, \tilde{x}_2, \tilde{x}_1^2 + \tilde{x}_2^2) \right\rangle_{\mathbb{F}} \\ &= x_1 \tilde{x}_1 + x_2 \tilde{x}_2 + (x_1^2 + x_2^2)(\tilde{x}_1^2 + \tilde{x}_2^2) \\ &= \langle \mathbf{x}, \tilde{\mathbf{x}} \rangle_{\mathbb{X}} + \|\mathbf{x}\|_{\mathbb{X}}^2 \cdot \|\tilde{\mathbf{x}}\|_{\mathbb{X}}^2 = k(\mathbf{x}, \tilde{\mathbf{x}}) \end{aligned}$$



Kernel Matrix / GRAM Matrix

We define:

Kernel matrix:

For a given dataset $\mathcal{D}$ comprising the N feature vectors $\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^N$, we define the **kernel matrix** (also known as **GRAM / GRAMian matrix**) as:

$$\mathbf{K} := [k_{ij}]_{i,j=1,2,\dots,N} \in \mathbb{R}^{N \times N} \quad (43)$$

where

$$k_{ij} := \left\langle \varphi(\mathbf{x}^i), \varphi(\mathbf{x}^j) \right\rangle_{\mathbb{F}} = k(\mathbf{x}^i, \mathbf{x}^j) \quad (44)$$

Kernel Matrix / GRAM Matrix (Ctd.)

Lemma: The kernel matrix $\mathbf{K}$ is positive semi-definite.

Proof: Let $\mathbf{z} \in \mathbb{R}^N$ be an arbitrary vector. Then we have

$$\begin{aligned} \mathbf{z}^\top \mathbf{K} \mathbf{z} &= \sum_{i=1}^N \sum_{j=1}^N z_i z_j k_{ij} \stackrel{(44)}{=} \sum_{i=1}^N \sum_{j=1}^N z_i z_j \left\langle \varphi(\mathbf{x}^i), \varphi(\mathbf{x}^j) \right\rangle_{\mathbb{F}} \\ &\stackrel{(6)}{=} \left\langle \sum_{i=1}^N z_i \varphi(\mathbf{x}^i), \sum_{j=1}^N z_j \varphi(\mathbf{x}^j) \right\rangle_{\mathbb{F}} = \left\| \sum_{i=1}^N z_i \varphi(\mathbf{x}^i) \right\|_{\mathbb{F}}^2 \geq 0. \end{aligned}$$



MERCER's Condition

- In practice, we are usually not interested in the mapping function φ itself
- We only need to know **that it exists**
- The mapping function φ is guaranteed to exist, if the kernel function k fulfills MERCER's condition (*see next slide*)

If the kernel function k satisfies MERCER's condition, we call k a **MERCER kernel**



MERCER's Condition (Ctd.)

MERCER's theorem:

For any **symmetric** function $k : \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{R}$ that is **square integrable** on its domain and which satisfies the condition

$$\int_{\mathbb{X}} \int_{\mathbb{X}} g(\mathbf{x}) g(\tilde{\mathbf{x}}) k(\mathbf{x}, \tilde{\mathbf{x}}) d\mathbf{x} d\tilde{\mathbf{x}} \geq 0 \quad (45)$$

for all square integrable functions g , there exist transforms $\varphi_j : \mathbb{X} \rightarrow \mathbb{R}$ and $\lambda_j \geq 0$ so that for all $\mathbf{x}, \tilde{\mathbf{x}} \in \mathbb{X}$

$$k(\mathbf{x}, \tilde{\mathbf{x}}) = \sum_j \lambda_j \varphi_j(\mathbf{x}) \varphi_j(\tilde{\mathbf{x}}). \quad (46)$$



When can Kernel Methods be used?

- Many algorithms can be '*kernelized*'
- The most prominent example are
 - Support Vector Machines,
 - Kernel regression,
 - Kernel Principal Component Analysis (PCA), and the
 - Kernel Perceptron

IMPORTANT: The prerequisite for using kernels is that the original input vectors appear exclusively in inner products. These inner products can then be replaced by a kernel function.



Well-known Kernels

- **Linear kernel**

$$k(\mathbf{x}, \tilde{\mathbf{x}}) = \langle \mathbf{x}, \tilde{\mathbf{x}} \rangle_{\mathbb{X}} \quad (47)$$

- **Polynomial kernel** (with hyperparameters $c \in \mathbb{R}$ and $p \in \mathbb{N}$)

$$k(\mathbf{x}, \tilde{\mathbf{x}}) = (\langle \mathbf{x}, \tilde{\mathbf{x}} \rangle_{\mathbb{X}} + c)^p \quad (48)$$

- **GAussian (RBF) kernel** (with hyperparameter $s > 0$, bandwidth parameter)

$$k(\mathbf{x}, \tilde{\mathbf{x}}) = \exp \left\{ -\frac{\|\mathbf{x} - \tilde{\mathbf{x}}\|_{\mathbb{X}}^2}{2s^2} \right\} \quad (49)$$



Kernels in other Domains

- The concept of kernels can be used in other domains as well:
- **String kernels**
 - Operate on strings
 - A string kernel measures the similarity of strings
 - String kernels allow kernel algorithms to work with strings **without having to translate these to fixed-length, real-valued feature vectors**
 - **p-spectrum kernels**: Count the number of substrings in common between the two strings
- **Graph kernels**

(This list is not exhaustive!)

Construction of new Kernels

In general it is not straightforward to check if MERCER's condition is satisfied, but it is possible to construct new kernels out of known ones:

Lemma: If $k_1(\mathbf{x}, \tilde{\mathbf{x}})$ and $k_2(\mathbf{x}, \tilde{\mathbf{x}})$ are valid kernels, then so are:

- $c \cdot k_1(\mathbf{x}, \tilde{\mathbf{x}})$ for any constant $c \in \mathbb{R}$
- $k_1(\mathbf{x}, \tilde{\mathbf{x}}) + k_2(\mathbf{x}, \tilde{\mathbf{x}})$
- $k_1(\mathbf{x}, \tilde{\mathbf{x}}) \cdot k_2(\mathbf{x}, \tilde{\mathbf{x}})$
- $f(\mathbf{x}) \cdot k_1(\mathbf{x}, \tilde{\mathbf{x}}) \cdot f(\tilde{\mathbf{x}})$ for any function f

This list is not exhaustive! See [Bishop.2006], chapter 6.2 for more details

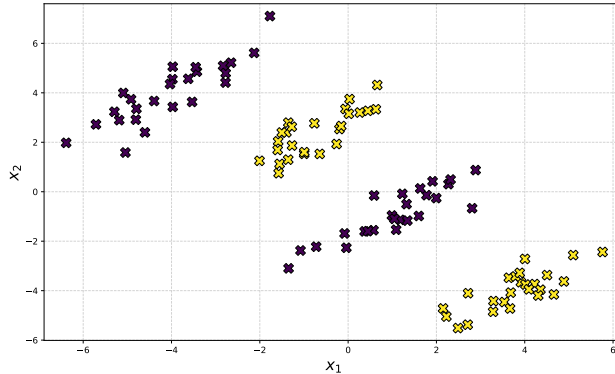
Incorporation of Kernel Functions for SVMs

- The kernel function k replaces any occurrence of a scalar product $\langle \mathbf{x}, \tilde{\mathbf{x}} \rangle_{\mathbb{X}}$ between feature vectors in the original space
- **Example:**

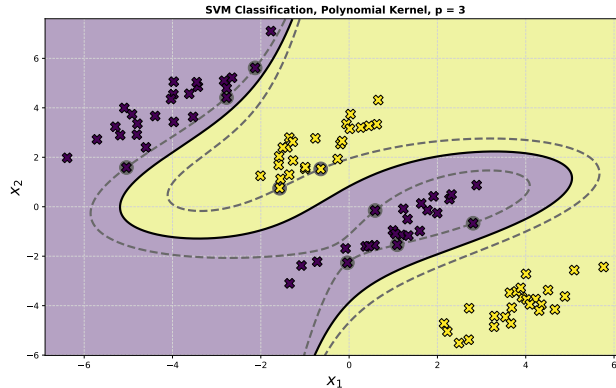
$$\mathcal{D}(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j k(\mathbf{x}^i, \mathbf{x}^j) \quad (50)$$

$$h(\mathbf{x}') = \text{sign} \left(\sum_{i \in \mathcal{S}} \alpha_i y_i k(\mathbf{x}^i, \mathbf{x}') + b \right) \quad (51)$$

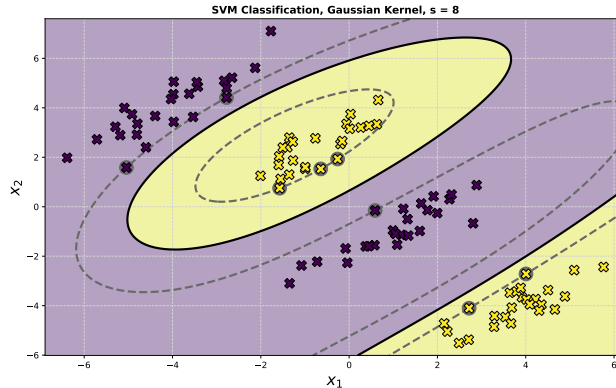
Example: Non-linear Data Set



Example: Polynomial Kernel ($p = 3$)



Example: GAUSSIAN (RBF) Kernel ($s = 8$)



Advantages and Disadvantages of the Kernel Method

Advantage: High-dimensional feature representations don't have to be computed explicitly! In theory we can work in **infinite-dimensional feature spaces!**

Disadvantages:

- The price is that we now have a **non-parametric method**, i. e. we need the training dataset to make predictions (*only the support vectors are needed!*)
- We no longer have explicit representations of the parameters $\mathbf{w}$ and b
- The method **becomes slow** for large amounts of data (> 50 k records)

Section:

Soft-Margin Support Vector Machines

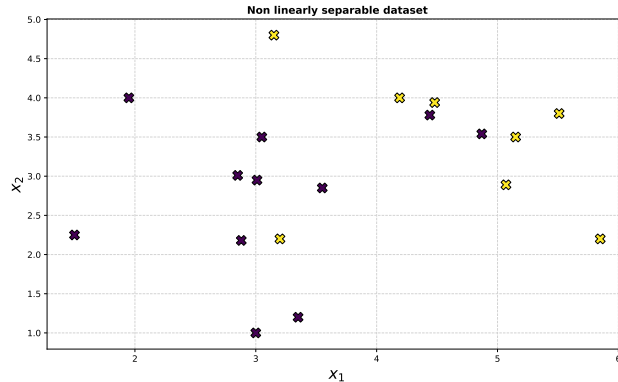
Overlapping Data
Slack Variables
Incorporating Slack for SVMs

Overlapping Distributions

- Until now we have assumed the data to be linearly separable
 $\implies$ **Hard-margin SVM finds the best separating hyperplane**
- **But:** In general, the classes may have overlapping distributions
 $\implies$ **Hard margin leads to overfitting and poor generalization**
- We will modify the algorithm to deal with overlapping distributions

Soft-margin SVMs allow for misclassifications of some data points while introducing a penalty. The penalty increases linearly with the distance from the decision boundary. This is done using **slack variables** ξ_n (German: *Schlupfvariable*).

Example Overlapping Class Distributions

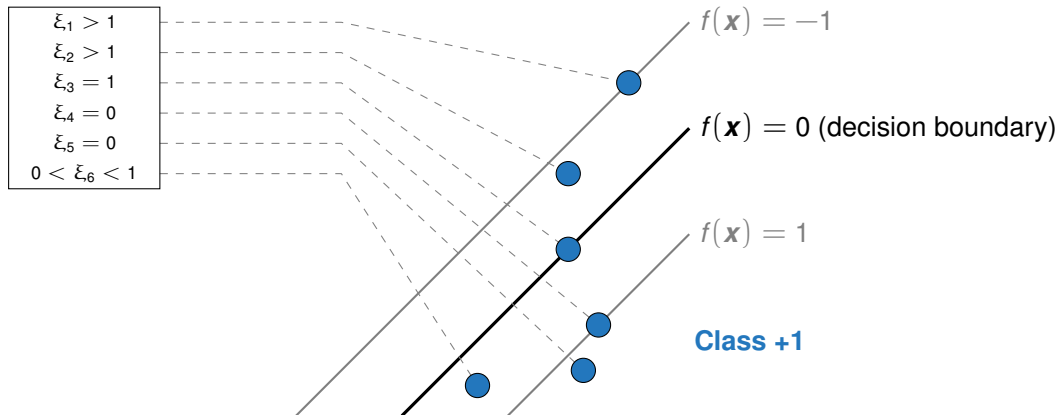


Slack Variables

- We introduce one slack variable $\xi_n \geq 0$ ($n = 1, \dots, N$) per data point
- **Different cases:**
 - $\xi_n = 0$ if $\mathbf{x}^n$ is on the correct side of the boundary
(*outside or on margin boundary*)
 - $0 < \xi_n < 1$ if $\mathbf{x}^n$ is inside the margin, but on the correct side of the boundary
 - $\xi_n = 1$ if $\mathbf{x}^n$ is on the decision boundary
 - $\xi_n > 1$ if $\mathbf{x}^n$ lies on the wrong side of the decision boundary (*misclassification*)
- The optimization constraints (28) are replaced with:

$$y_n f(\mathbf{x}^n) \geq 1 - \xi_n \quad (n = 1, \dots, N) \quad (52)$$

Slack Variables (Ctd.)



Primal Optimization Problem for Soft-Margin SVMs

Maximize the margin, but penalize points on the wrong side of the margin boundary

Primal optimization problem for soft-margin SVMs:

$$\text{minimize} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n \quad (C > 0) \quad (53)$$

$$\text{subject to} \quad y_n (\langle \mathbf{w}, \mathbf{x}^n \rangle + b) \geq 1 - \xi_n \quad (54)$$

$$\xi_n \geq 0 \quad (n = 1, \dots, N) \quad (55)$$

Corresponding LAGRANGE Function

- $C > 0$ is a hyperparameter of the model controlling the **degree of softness**
- The larger C the more we penalize
- $C \rightarrow \infty$ recovers the hard-margin SVM introduced in the first section
- The LAGRANGE function is: $(\alpha, \beta \in \mathbb{R}^N$ are vectors of LAGRANGE multipliers)

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b, \xi, \alpha, \beta) = & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n \\ & - \sum_{n=1}^N \alpha_n [y_n f(\mathbf{x}^n) - 1 + \xi_n] - \sum_{n=1}^N \beta_n \xi_n \end{aligned} \quad (56)$$

Dual Optimization Problem for Soft-Margin SVMs

Dual optimization problem for soft-margin SVMs:

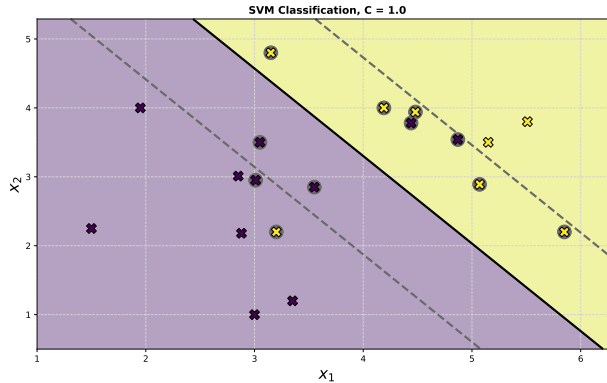
$$\text{maximize} \quad \mathcal{D}(\boldsymbol{\alpha}) := \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \langle \mathbf{x}^i, \mathbf{x}^j \rangle \quad (57)$$

$$\text{subject to} \quad 0 \leq \alpha_i \leq C \quad (i = 1, \dots, N) \quad (58)$$

$$\sum_{i=1}^N \alpha_i y_i = 0 \quad (59)$$

The constraints in (58) are called **box constraints** (for obvious reasons)

Example: Soft Margin SVM



Section:

Sequential Minimal Optimization (SMO)

Introduction to SMO

Computation of the Bounds L and H

Maximization of the Objective

SMO Termination Criterion

Alternative: Learning a linear SVM using `cvxopt`

What is SMO?

- **Sequential Minimal Optimization (SMO)** is an optimization algorithm for solving the quadratic programming problem that arises during the training of SVMs
- It uses a modified version of **coordinate ascent**
- SMO was introduced 1998 by JOHN PLATT at Microsoft Research
- Please find the original paper $\Rightarrow$ [here](#)

Why the name? Rather than solving the problem numerically as a whole, SMO breaks it into a series of smallest possible sub-problems which are solved analytically in a sequential manner

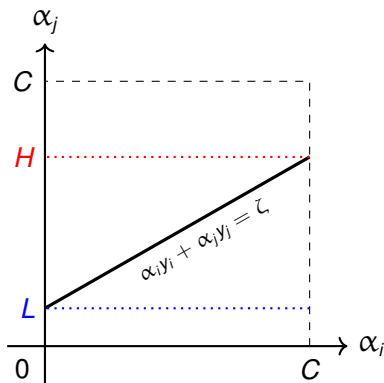
Main Idea of the Algorithm

- At each iteration, SMO ensures that the constraints (58) and (59) are satisfied
- SMO updates **two multipliers at a time** (*the others are fixed*)
- Rearranging (59) for α_i reveals a **functional dependence** of α_i on the other α 's

$$\alpha_i = -y_i \sum_{\substack{k=1 \\ k \neq i}}^N \alpha_k y_k \quad (60)$$

- This means we cannot change α_i without changing at least one other multiplier α_j , otherwise constraint (59) is violated

Main Idea of the Algorithm (Ctd.)



- We choose to update α_i and α_j for $i \neq j$
- From (59) we obtain:

$$\alpha_i y_i + \alpha_j y_j = - \sum_{\substack{k=1 \\ k \neq i, j}}^N \alpha_k y_k =: \zeta \quad (61)$$

- The solution has to lie on the line so as to fulfill the constraint
- We have to choose $\alpha_j \in [L, H]$

Computation of L and H

- We begin by expressing α_i in terms of α_j : ($\Rightarrow 1/y_i = y_i$, as $y_i \in \{-1, 1\}$)

$$\alpha_i y_i + \alpha_j y_j = \zeta \quad \Longleftrightarrow \quad \alpha_i = y_i (\zeta - \alpha_j y_j) \quad (62)$$

- We know $0 \leq \alpha_i \leq C$, therefore $0 \leq y_i (\zeta - \alpha_j y_j) \leq C$
- Rearranging for α_j we obtain:

$$y_i \zeta - C \leq \alpha_j \leq y_i \zeta \quad \text{if } y_i = y_j \quad (63)$$

$$-y_i \zeta \leq \alpha_j \leq C - y_i \zeta \quad \text{if } y_i \neq y_j \quad (64)$$

Computation of L and H (Ctd.)

- We plug in the definition of ζ (61) to simplify these bounds:

$$\alpha_i^{\text{old}} + \alpha_j^{\text{old}} - C \leq \alpha_j \leq \alpha_i^{\text{old}} + \alpha_j^{\text{old}} \quad \text{if } y_i = y_j \quad (65)$$

$$\alpha_j^{\text{old}} - \alpha_i^{\text{old}} \leq \alpha_j \leq C + \alpha_j^{\text{old}} - \alpha_i^{\text{old}} \quad \text{if } y_i \neq y_j \quad (66)$$

- Also we know that $0 \leq \alpha_j \leq C$ has to be fulfilled
- We therefore have two lower bounds and two upper bounds for α_j which have to be satisfied **simultaneously**

Computation of L and H (Ctd.)

We combine the conditions for α_j to obtain:

Case 1: $y_i = y_j$

$$L := \max(0, \alpha_i^{\text{old}} + \alpha_j^{\text{old}} - C)$$

$$H := \min(C, \alpha_i^{\text{old}} + \alpha_j^{\text{old}})$$

Case 2: $y_i \neq y_j$

$$L := \max(0, \alpha_j^{\text{old}} - \alpha_i^{\text{old}})$$

$$H := \min(C, C + \alpha_j^{\text{old}} - \alpha_i^{\text{old}})$$

We have: We know the range α_j has to be chosen from.

We need: As a next step we have to find the maximum on the line!

Intuition: Maximization of the Objective

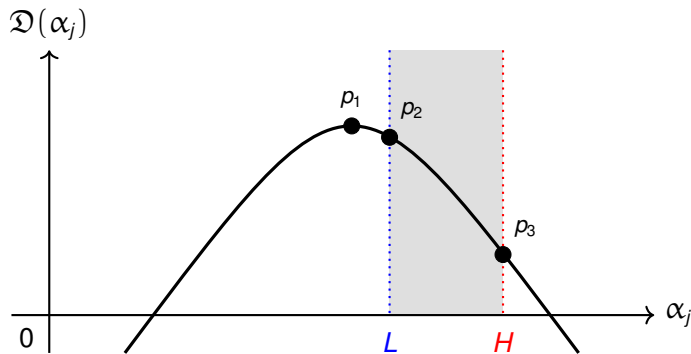
- We only update the parameters α_i and α_j (*all other α 's are fixed*) and use equation (62) to express α_i in terms of α_j
- The objective function (57) becomes a **1-dimensional quadratic function** in α_j :

$$\begin{aligned}\mathfrak{D}(\alpha_i, \alpha_j) &= \mathfrak{D}(y_i(\zeta - \alpha_j y_j), \alpha_j) \\ &= a\alpha_j^2 + b\alpha_j + c\end{aligned}\tag{67}$$

for some constants $a, b, c \in \mathbb{R}$ (*where the constant c can be ignored*)

This problem is easy to solve with standard optimization techniques. But consider that the optimal value for α_j has to lie within the bounds L and H !

Intuition: Maximization of the Objective (Ctd.)



- We only have to check the three points p_1, p_2, p_3
- Left: The global optimum is not in $[L, H]$
- We have to **clip** the new value for α_j
- We set $\alpha_j^{\text{new}} := L$

Maximization of the Objective: Formulas

- The value for α_j at the global optimum p_1 is given by (*check the paper for details!*):

$$\alpha_j^* := \alpha_j^{\text{old}} - \frac{y_j(E_i - E_j)}{\eta} \quad (68)$$

$$E_k := \sum_{n=1}^N \alpha_n y^n \langle \mathbf{x}^n, \mathbf{x}^k \rangle + b - y_k \quad (69)$$

$$\eta := 2\langle \mathbf{x}^i, \mathbf{x}^j \rangle - \langle \mathbf{x}^i, \mathbf{x}^i \rangle - \langle \mathbf{x}^j, \mathbf{x}^j \rangle \quad (70)$$

- E_k is the error between the SVM output on the k -th example and the true label

Maximization of the Objective: Formulas (Ctd.)

- η is the second-order derivative of $\mathcal{D}$ along the line
- When calculating η you may want to **replace the inner products with kernel functions**
- Next, we clip α_j to lie within the range $[L, H]$:

$$\alpha_j^{\text{new}} := \begin{cases} H & \text{if } \alpha_j^* > H \\ \alpha_j^* & \text{if } L \leq \alpha_j^* \leq H \\ L & \text{if } \alpha_j^* < L \end{cases} \quad (71)$$

Computation of α_i^{new}

- We have to compute α_i^{new} based on the value α_j^{new} to satisfy the constraint (59)
- We had: $\alpha_i = y_i(\zeta - \alpha_j y_j)$, therefore we get the update rule:

$$\begin{aligned}\alpha_i^{\text{new}} &= y_i(\zeta - \alpha_j^{\text{new}} y_j) \\ &= y_i(\alpha_i^{\text{old}} y_i + \alpha_j^{\text{old}} y_j - \alpha_j^{\text{new}} y_j) \\ &= \alpha_i^{\text{old}} + y_i y_j (\alpha_j^{\text{old}} - \alpha_j^{\text{new}})\end{aligned}\tag{72}$$

KARUSH-KUHN-TUCKER Conditions for the SVM

- The **KARUSH-KUHN-TUCKER (KKT) conditions** provide a necessary condition for the optimal solution
- For SVMs, these conditions are also sufficient:

$$\alpha_n = 0 \implies y_n(\langle \mathbf{w}, \mathbf{x}^n \rangle + b) \geq 1 \quad (73)$$

$$\alpha_n = C \implies y_n(\langle \mathbf{w}, \mathbf{x}^n \rangle + b) \leq 1 \quad (74)$$

$$0 < \alpha_i < C \implies y_n(\langle \mathbf{w}, \mathbf{x}^n \rangle + b) = 1 \quad (75)$$

- Iterate until these conditions are met (*to within some numerical tolerance*)

What is `cvxopt`?

- `cvxopt` is a Python package for **convex optimization**
- Please find the documentation of the package $\Rightarrow$ [here](#)
- Let's see how it works for linear SVMs

Rewriting the Optimization Problem

Recall our original optimization problem:

maximize

$$\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}^i, \mathbf{x}^j \rangle$$

subject to

$$\alpha_i \geq 0 \quad (i = 1, \dots, N)$$

$$\sum_{i=1}^N \alpha_i y^{(i)} = 0$$

`cvxopt` requires a different format:

$$\text{minimize} \quad \frac{1}{2} \mathbf{x}^\top \mathbf{P} \mathbf{x} + \mathbf{q}^\top \mathbf{x} \quad (76)$$

$$\text{subject to} \quad \mathbf{A} \mathbf{x} = \mathbf{b} \quad (77)$$

$$\mathbf{G} \mathbf{x} \leq \mathbf{h} \quad (78)$$

Rewriting the Optimization Problem (Ctd.)

We multiply the objective by -1 to obtain a **minimization problem** and define:

- Let $\mathbf{P} \in \mathbb{R}^{N \times N}$ be given by $[p_{ij}]_{i,j=1,2,\dots,N}$ with

$$p_{ij} := y_i y_j \langle \mathbf{x}^i, \mathbf{x}^j \rangle$$

- $\boldsymbol{\alpha} \in \mathbb{R}^N$ (vector of LAGRANGE multipliers)
- $\mathbf{q} \in \mathbb{R}^N$ is a constant vector whose components are equal to 1 (i. e. $\mathbf{q} := \mathbf{1}$)
- $\mathbf{A} \in \mathbb{R}^N$ is the vector containing the labels ($\mathbf{A} := \mathbf{y}$); $\mathbf{b} \in \mathbb{R}$, $\mathbf{b} := 0$
- $\mathbf{G} := -\mathbf{I}_N$ is the negative $N \times N$ identity matrix
- $\mathbf{h} \in \mathbb{R}^N$ is a constant vector whose components are equal to zero (i. e. $\mathbf{h} := \mathbf{0}$)

Rewriting the Optimization Problem (Ctd.)

- Therefore, we get the following optimization problem:

$$\text{minimize} \quad \frac{1}{2} \boldsymbol{\alpha}^\top \mathbf{P} \boldsymbol{\alpha} + \mathbf{1}^\top \boldsymbol{\alpha} \quad (79)$$

$$\text{subject to} \quad \mathbf{y}^\top \mathbf{x} = 0 \quad (80)$$

$$-\boldsymbol{\alpha} \leq \mathbf{0} \quad (81)$$

- Call of the library function:

```
sol = solvers.qp(P, q, G, h, A, b)
alphas = np.array(sol["x"]) ← LAGRANGE multipliers
```

Section: Wrap-Up

Summary
Recommended Literature
Self-Test Questions
Lecture Outlook

Summary

- Standard SVMs assume the data to be **linearly separable**
- **Generalization guarantee:** SVMs are **maximum margin** classifiers
- The set of **support vectors** defines the decision boundary
- We have to solve a quadratic optimization problem to obtain the support vectors which are needed for prediction
- **Kernels** enable SVMs to classify non-linear data **without having to compute explicit representations** of the data in the high-dimensional feature space
- Slack variables allow for **soft-margin classification**
- **SMO** is an efficient algorithm to solve the quadratic programming problem

Recommended Literature

- 1 [BISHOP.2006], chapter 6
- 2 [BISHOP.2006], chapter 7
- 3 [MURPHY, 2012], chapter 14

(For free PDF versions, see list in GitHub readme!)



Self-Test Questions

- 1 What is a maximum-margin classifier? What advantages does it have compared to other classifiers?
- 2 Which data points are needed for prediction? How do we get them?
- 3 What is a kernel? Can every function serve as a kernel?
- 4 What prerequisite must be fulfilled so that kernels can be used?
- 5 Name well-known kernels and write down the equation to compute them!
- 6 What is a slack variable? What can we do with it?
- 7 Outline the SMO algorithm!

What's next...?

- | | | | |
|-------------|-----------------------------------|--------------|------------------------------|
| I | Machine Learning Introduction | IX | Evaluation |
| II | Optimization Techniques | X | Decision Trees |
| III | Bayesian Decision Theory | XI | Support Vector Machines |
| IV | Non-parametric Density Estimation | • XII | Clustering |
| V | Probabilistic Graphical Models | XIII | Principal Component Analysis |
| VI | Linear Regression | XIV | Reinforcement Learning |
| VII | Logistic Regression | XV | Advanced Regression |
| VIII | Deep Learning | | |

Thank you very much for the attention!

***** Artificial Intelligence and Machine Learning *****

Topic: Support Vector Machines (SVMs) and Kernel Methods

Term: Summer term 2026

Contact:

Daniel Wehner, M.Sc.

SAP SE / DHBW CAS Heilbronn

daniel.wehner@sap.com

Do you have any questions?